Lima Pilar Konsep Object-Oriented

1. **Encapsulation dan Information Hiding.**
2. **Inheritance.**
3. **Overloading.**
4. **Overriding dan Polymorphism.**
5. **Multithreading.**

Encapsulation dan Information Hiding

- Class perlu bekerjasama dengan class yg lain dalam suatu project, namun sebaliknya ada kalanya suatu class tidak diperlukan class lain untuk diakses.
- Encapsulation adalah cara agar elemen/ komponen secara anatomy milik suatu class ***hanya dapat diakses*** oleh class pemiliknya (tidak boleh diakses oleh class lain).
- Information hiding adalah ***tujuan*** dari encapsulation (bersifat rahasia: menyembunyikan informasi yg memang tidak diperlukan untuk diketahui secara umum).
- Cara encapsulation yaitu **attribute** maupun ***method()*** atau ***constructor()*** disembunyikan dengan memberikan modifier ***private***.

Pengertian Inheritance (Pewarisan)

Ciri-ciri khas (attribute) dan/atau perilaku yang khas (method()) milik *super class* diwariskan ke ***subclass***, sehingga subclass boleh memanfaatkan komponen superclass.

Konsep ini menyusun struktur/hirarki taxonomy class-class suatu project.

Contoh dalam hirarki class, jika class JuringBola merupakan turunan class Bola, dan class Bola merupakan turunan class Lingkaran, maka otomatis attribute dan method() atau constructor() milik super class Lingkaran, juga diwariskan ke sub class Bola dan ke sub class JuringBola.

Template pewarisan:

```
[modifier1] class NamaSubClass extends NamaSuperClass {
    //classbody
}
```

Manfaat Inheritance (Pewarisan)

- **Tanpa** *inheritance*, maka seluruh *attribute* dan *method()* atau *constructor()* yang dicoding di suatu *class*, **harus ditulis ulang seluruhnya** oleh suatu *class* lain yang kebetulan **memerlukan** sebagian code program yang telah ditulis.
- **Dengan** *inheritance,* berarti code yang dibutuhkan **tidak perlu ditulis ulang**.
- **Dengan** *inheritance*, jika programmer ingin memanfaatkan suatu *attribute* atau memodifikasi isi suatu *method()* milik *super class*, dapat dilakukan di *sub class*.
- Namun bila logika dari *coding* ingin diperbaiki maka prosesnya dapat dilakukan di *super class*.

Overloading terhadap method() dan constructor()

- ***Overloading*** menggambarkan banyaknya cara untuk melakukan sesuatu dengan membuat method() sebanyak caranya dengan **nama** yang **sama** dalam class yang **sama**, namun harus dengan memodifikasi deklarasi parameter. Contohnya method pangkat(int x, int y); pangkat(int x, float y); pangkat(float x, int y); pangkat(float x, float y);

- Method-method yg ***overloading*** wajib berada di dalam class yg sama atau dalam satu keluarga (sebagian method()-method() tersebut di dalam superclass dan yg lain di dalam subclass). Jika type parameter tidak diganti di subclass-nya maka overloading berubah menjadi overriding.

- Overloading di OOP berbeda dengan overloading terhadap ***operator*** di C++.

- Interpreter/compiler mampu memilih method() mana yang dieksekusi dengan mengenali **tipe parameter** (input) yg dilewatkan ke method() dan mengenali **return value**nya (output).

Overriding attribute, constructor() dan method()

- **Overriding** menggambarkan bagaimana detail cara suatu pekerjaan dilaksanakan, overriding adalah kemampuan suatu **subclass** untuk menulis code program yg lebih rinci di method() yang abstract **milik super class**, atau memodifikasi (mengubah cara) isi method() atau isi attribute yang tidak abstract, tidak private, tidak final, dan non static **milik super class**.
- Modifikasi modifier-akses pada attribute, method(), dan constructor() **milik super class**, **boleh dilakukan** di subclass dengan cara **memperluas daerah aksesnya**.

Overriding attribute, constructor() dan method()

- Misal modifier-akses suatu attribute, method(), atau constructor() di super class adalah **protected**, maka modifier akses tersebut dapat diperluas menjadi **public** di subclass nya.
- **Argument** atau **tipe parameter** milik method() dari suatu super class pada overriding **tidak boleh dimodifikasi** oleh sub class nya, bila dilakukan maka berubah menjadi **overloading**.
- **Tipe return value** pada method() atau constructor() milik super class, **tidak boleh diubah** oleh sub class nya.

```
modifier1] class NamaSubClass implemets NamaInterface {
    //classbody
}
```

Overriding vs *implementation*

- method() abstract milik super class, wajib di overriding dalam subclass nya, mirip dengan implementation terhadap method() milik Interface. Karena semua method() di Interface bersifat abstract, sehingga semua method() nya harus diisi pada class implementasinya.

Menggunakan/tulis ulang isi method() dan constructor() milik super class

- Subclass tidak diperkenankan untuk memanggil constructor() milik super class dengan langsung memanggil **namanya.**
- Cara yang benar: ***super()*** atau ***super(arguments);***
- Jika terjadi overriding berarti ada method() milik super class yang digunakan atau ditulis ulang di sub class nya.
- Cara pemanggilan method() yang **non static** milik super class: ***super.namaMethod(tipe parameter);***
- Cara pemanggilan attribute yang **non static** milik super class: ***super.namaAttribute;***

Polymorphism

- **Polymorphism arti bahasanya bersifat banyak-bentuk. Biasanya dilakukan di Main Class.**

- **Konsep ini sangat terkait dengan konsep Inheritance, jika tidak terjadi inheritance maka polymorphism juga tidak ada.**

- **Super Class yang sama dapat mewujudkan banyak objek yang dapat memiliki literal (nilai) yang berbeda-beda sesuai dengan constructor() milik Subclass nya.**

- **Jadi, super class yang sama dapat memiliki banyak perilaku dengan keluaran yang berbeda-beda sesuai dengan constructor subclassnya.**

- **Contoh:**
- ***Parent p = new Parent();***
- ***Parent p1 = new Child1();***
- ***Parent p2 = new Child2();***
- ***Parent p3 = new Child3();***
- **Perhatikan bahwa Super Class nya sama yaitu *Parent*.**

```
public class Parent {

}
public class Child1 extends Parent {

}
public class Child2 extends Parent {

}
public class Child3 extends Parent {

}

public class Polymohism {
    main() {
        Parent p = new Parent();
        Parent p1 = new Child1();
        Parent p2 = new Child2();
        Parent p3 = new Child3();
    }
    /** objek p1 ini dapat memanggil method()-method() milik
    Parent maupun Child1 */
}
```

```
public class Manusia {
    public String name;
    public String alamat;

    public void berjalan() {

    }
    public String getName() {
        return name;
    }
}
```

pDog, Class Dog.java

```
public class Dog {
   // information hidding
   private int weight;
  // method jenis getter()
   public int getWeight() {
      return weight;
   }
  // method jenis setter()
   public void setWeight(int newWeight) {
      weight = newWeight;
  }
}
```

pDog, Class TestDog.java

- Dan sebuah class TesDog yang memanfaatkan class Dog tersebut:

```
public class TesDog {
 public static void main(String[] args) {
    Dog bleki = new Dog();
      // menggunakan method setter()
      bleki.setWeight(42);
      //menggunakan method getter()
      System.out.println("Dog's weight is
  "+bleki.getWeight());
  }
}
```

Output:
Dog's weight is 42

pEmployee, Class Employee.java

```
public class Employee extends Manusia {
    //private String name;
    private double salary;
    protected Employee(String n, double s) {
        super.name=n;
        salary = s;

    }
    protected String getDetails() {
        return    "Name:    "+super.name+    "\nSalary:
  "+salary;
    }
    public void cetak() {
        System.out.println("Coba di Employee");
    }

}
```

Class Manager.java

```
public class Manager extends Employee {
    private String department;
    //private double tunJab;
    public Manager(String nama, double salary, String dep) {
        super(nama, salary);
        department = dep;
    }
    public String getDepartment() {
        return department;
    }
    public String getDetails() {
        return super.getDetails()+"\nDepartment:
  "+getDepartment();
    }
    public void cetak() {
        System.out.println("Coba di Manager");
    }
}
```

Class View.java

```
Public class View {
    public static void main(String[] args) {
        Employee e = new Employee("Ali",12000000);
        Employee em;
        em = new  Manager("Ali",12000000,"Informatika");
        System.out.println("Data employee:\n"+e.getDetails());
        System.out.println("Data manager:\n"+em.getDetails());
        e.cetak();
        em.cetak();
    }
}
```

Output:

```
Data employee :
Name: Ali
Salary: 1200000.0
Data manager:
Name: Ali
Salary: 1200000.0
Department: Informatika
Coba di Manager
```

Catatan:

Kalau method cetak() di class Employee dan class Manager ada, maka yang dijalankan adalah method milik class Manager. Prioritasnya adalah class Manager kemudian class Employee.

Class Manusia.java

```
public class Manusia {
    public String nama;
    public String alamat;

    public void berjalan() {

    }
    public String getNama() {
        return nama;
    }
}
```

Class Pegawai.java

```
public class Pegawai extends
Manusia {
    public int nip;
    public int gaji;
    public int ambilGaji() {
        return gaji;
    }
 }
```

pEkspresiWajah, Class EkspresiWajah.java

```
class EkspresiWajah{
    public String respons() {
        return("Perhatikan ekspresi wajah saya");
    }
}
```

Class-Class turunan EkspresiWajah.java

```
class WajahGembira extends EkspresiWajah {
    public String respons() {
        return("ha ha ha..");}
}
class WajahSedih extends EkspresiWajah {
    public String respons() {
        return("hik hik ngeee ngeee ngeee..");}
}
class WajahMarah extends EkspresiWajah {
    public String respons() {
        return("Hai kurang ajar..!");}
}
```

Class MainEkspresiWajah.java

```
Class MainEkspresiWajah{
 public static void main(String args[]) {
   EkspresiWajah objEkspresi = new EkspresiWajah();
   EkspresiWajah objGembira = new WajahGembira();
   EkspresiWajah objSedih = new WajahSedih();
   EkspresiWajah objMarah = new WajahMarah();

   EkspresiWajah[] arrEkspresi = new EkspresiWajah[4];
   arrEkspresi[0] = objEkspresi;
   arrEkspresi[1] = objGembira;
   arrEkspresi[2] = objSedih;
   arrEkspresi[3] = objMarah;

   System.out.println("Ekspresi[0]: "+arrEkspresi[0].respons());
   System.out.println("Ekspresi[1]: "+arrEkspresi[1].respons());
   System.out.println("Ekspresi[2]: "+arrEkspresi[2].respons());
   System.out.println("Ekspresi[3]: "+arrEkspresi[3].respons());
    }
}
```